Topic Modeling with Scikit Learn

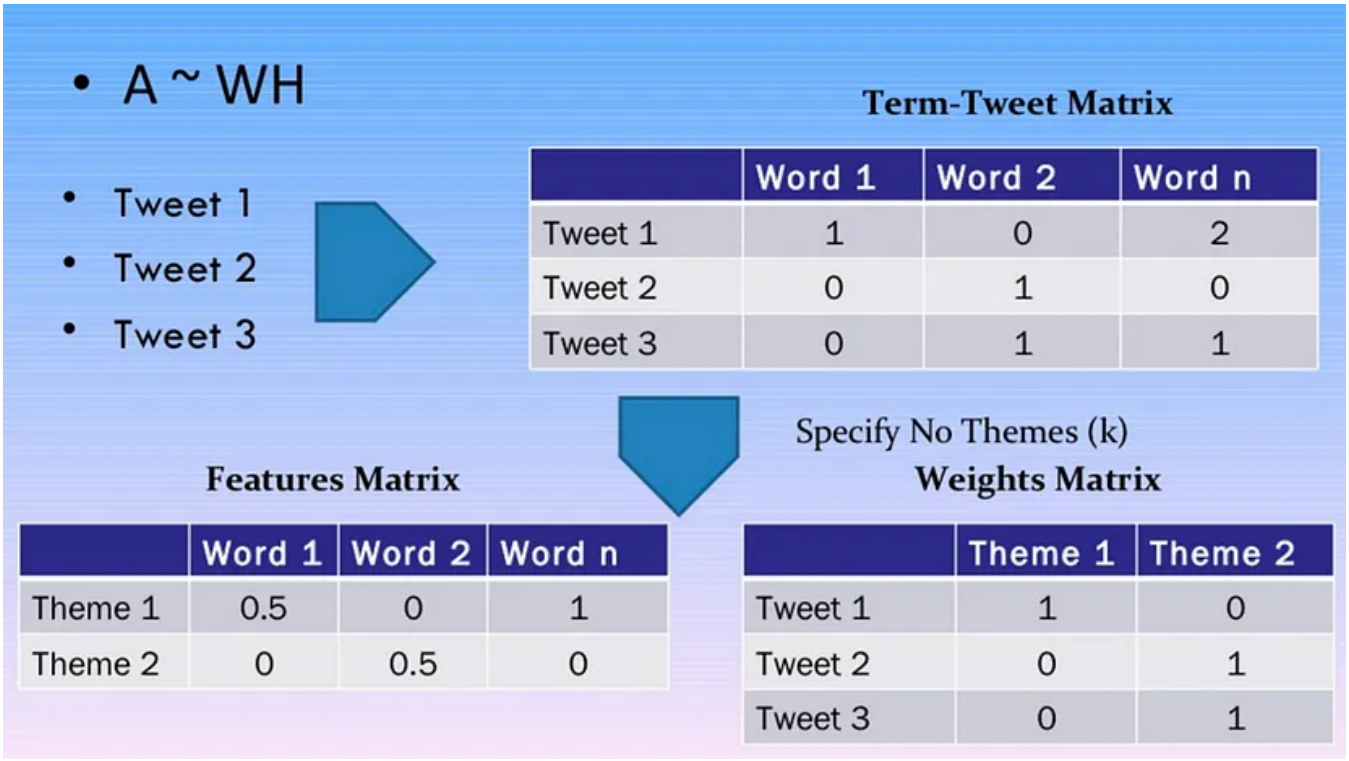
Aneesha Bakharia · Follow
Published in ML Review
5 min read · Sep 1, 2016

Listen Share

Latent Dirichlet Allocation (LDA) is a algorithms used to discover the topics that are present in a corpus. A few open source libraries exist, but if you are using Python then the main contender is Gensim. Gensim is an awesome library and scales really well to large text corpuses. Gensim, however does not include Non-negative Matrix Factorization (NMF), which can also be used to find topics in text. The mathematical basis underpinning NMF is quite different from LDA. I have found it interesting to compare the results of both of the algorithms and have found that NMF sometimes produces more meaningful topics for smaller datasets. NMF has been included in Scikit Learn for quite a while but LDA has only recently (late 2015) been included. The great thing about using Scikit Learn is that it brings API consistency which makes it almost trivial to perform Topic Modeling using both LDA and NMF. Scikit Learn also includes seeding options for NMF which greatly helps with algorithm convergence and offers both online and batch variants of LDA.

How do LDA and NMF work?

I won't go into any lengthy mathematical detail — there are many blogs posts and academic journal articles that do. While LDA and NMF have differing mathematical underpinning, both algorithms are able to return the documents that belong to a topic in a corpus and the words that belong to a topic. LDA is based on probabilistic graphical modeling while NMF relies on linear algebra. Both algorithms take as input a bag of words matrix (i.e., each document represented as a row, with each columns containing the count of words in the corpus). The aim of each algorithm is then to produce 2 smaller matrices; a document to topic matrix and a word to topic matrix that when multiplied together reproduce the bag of words matrix with the lowest error.



The resulting matrices from the NMF algorithm

> *No magic here — you need to specify the number of topics!*

How many topics? Well that is the question! Both NMF and LDA are not able to automatically determine the number of topics and this must be specified.

Dataset Preprocessing

I searched far and wide for an exciting dataset and finally selected the 20 Newsgoups dataset. I'm just being sarcastic — I selected a dataset that is both easy to interpret and load in Scikit Learn. The dataset is easy to interpret because the 20 Newsgroups are known and the generated topics can be compared to the known topics being discussed. Headers, footers and quotes are excluded from the dataset.

```
1  from sklearn.datasets import fetch_20newsgroups
2
3  dataset = fetch_20newsgroups(shuffle=True, random_state=1, remove=('headers', 'footers', 'quotes
4  documents = dataset.data
```

load20newsgroups.py hosted with ❤ by GitHub view raw

The creation of the bag of words matrix is very easy in Scikit Learn — all the heavy lifting is done by the feature extraction functionality provided for text datasets. A tf-idf transformer is applied to the bag of words matrix that NMF must process with the TfidfVectorizer. LDA on the other hand, being a probabilistic graphical model

(i.e. dealing with probabilities) only requires raw counts, so a CountVectorizer is used. Stop words are removed and the number of terms included in the bag of words matrix is restricted to the top 1000.

```
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer

no_features = 1000

# NMF is able to use tf-idf
tfidf_vectorizer = TfidfVectorizer(max_df=0.95, min_df=2, max_features=no_features, stop_words=
tfidf = tfidf_vectorizer.fit_transform(documents)
tfidf_feature_names = tfidf_vectorizer.get_feature_names()

# LDA can only use raw term counts for LDA because it is a probabilistic graphical model
tf_vectorizer = CountVectorizer(max_df=0.95, min_df=2, max_features=no_features, stop_words='en
tf = tf_vectorizer.fit_transform(documents)
tf_feature_names = tf_vectorizer.get_feature_names()
```

preprocess.py hosted with ❤ by GitHub view raw

NMF and LDA with Scikit Learn

As mentioned previously the algorithms are not able to automatically determine the number of topics and this value must be set when running the algorithm. Comprehensive documentation on available parameters is available for both NMF and LDA. Initialising the W and H matrices in NMF with 'nndsvd' rather than random initialisation improves the time it takes for NMF to converge. LDA can also be set to run in either batch or online mode.

```
from sklearn.decomposition import NMF, LatentDirichletAllocation

no_topics = 20

# Run NMF
nmf = NMF(n_components=no_topics, random_state=1, alpha=.1, l1_ratio=.5, init='nndsvd').fit(tfid
# Run LDA
lda = LatentDirichletAllocation(n_topics=no_topics, max_iter=5, learning_method='online', learni
```

nmf_lda_scikitlearn.py hosted with ❤ by GitHub view raw

Displaying and Evaluating Topics

The structure of the resulting matrices returned by both NMF and LDA is the same and the Scikit Learn interface to access the returned matrices is also the same. This is great and allows for a common Python method that is able to display the top words in a topic. Topics are not labeled by the algorithm — a numeric index is assigned.

```
def display_topics(model, feature_names, no_top_words):
    for topic_idx, topic in enumerate(model.components_):
        print "Topic %d:" % (topic_idx)
        print " ".join([feature_names[i]
                        for i in topic.argsort()[:-no_top_words - 1:-1]])

no_top_words = 10
display_topics(nmf, tfidf_feature_names, no_top_words)
display_topics(lda, tf_feature_names, no_top_words)
```

displaytopics.py hosted with ❤ by GitHub view raw

The derived topics from NMF and LDA are displayed below. From the NMF derived topics, Topic 0 and 8 don't seem to be about anything in particular but the other topics can be interpreted based upon there top words. LDA for the 20 Newsgroups dataset produces 2 topics with noisy data (i.e., Topic 4 and 7) and also some topics that are hard to interpret (i.e., Topic 3 and Topic 9). I'd say the NMF was able to find more meaningful topics in the 20 Newsgroups dataset.

NMF Topics:
Topic 0: people don think like know time right good did say
Topic 1: windows file use dos files window using program problem card
Topic 2: god jesus bible christ faith believe christian christians church sin
Topic 3: drive scsi drives hard disk ide controller floppy cd mac
Topic 4: game team year games season players play hockey win player
Topic 5: key chip encryption clipper keys government escrow public use algorithm
Topic 6: thanks does know mail advance hi anybody info looking help
Topic 7: car new 00 sale price 10 offer condition shipping 20
Topic 8: just like don thought ll got oh tell mean fine
Topic 9: edu soon cs university com email internet article ftp send

LDA Topics:
Topic 0: government people mr law gun state president states public use
Topic 1: drive card disk bit scsi use mac memory thanks pc

Topic 2: said people armenian armenians turkish did saw went came women
Topic 3: year good just time game car team years like think
Topic 4: 10 00 15 25 12 11 20 14 17 16
Topic 5: windows window program version file dos use files available display
Topic 6: edu file space com information mail data send available program
Topic 7: ax max b8f g9v a86 pl 145 1d9 0t 34u
Topic 8: god people jesus believe does say think israel christian true
Topic 9: don know like just think ve want does use good

In my next blog post, I'll discuss topic interpretation and show how top documents within a theme can also be displayed.

Full Code Listing

It's amazing how much can be achieved with just 36 lines of Python code and some Scikit Learn magic. The full code listing is provided below:

```
from sklearn.feature_extraction.text import TfidfVectorizer, CountVectorizer
from sklearn.datasets import fetch_20newsgroups
from sklearn.decomposition import NMF, LatentDirichletAllocation

def display_topics(model, feature_names, no_top_words):
    for topic_idx, topic in enumerate(model.components_):
        print "Topic %d:" % (topic_idx)
        print " ".join([feature_names[i]
                        for i in topic.argsort()[:-no_top_words - 1:-1]])

dataset = fetch_20newsgroups(shuffle=True, random_state=1, remove=('headers', 'footers', 'quote
documents = dataset.data

no_features = 1000

# NMF is able to use tf-idf
tfidf_vectorizer = TfidfVectorizer(max_df=0.95, min_df=2, max_features=no_features, stop_words=
tfidf = tfidf_vectorizer.fit_transform(documents)
tfidf_feature_names = tfidf_vectorizer.get_feature_names()

# LDA can only use raw term counts for LDA because it is a probabilistic graphical model
tf_vectorizer = CountVectorizer(max_df=0.95, min_df=2, max_features=no_features, stop_words='en
tf = tf_vectorizer.fit_transform(documents)
tf_feature_names = tf_vectorizer.get_feature_names()

no_topics = 20

# Run NMF
nmf = NMF(n_components=no_topics, random_state=1, alpha=.1, l1_ratio=.5, init='nndsvd').fit(tfi

# Run LDA
lda = LatentDirichletAllocation(n_topics=no_topics, max_iter=5, learning_method='online', learn

no_top_words = 10
display_topics(nmf, tfidf_feature_names, no_top_words)
display_topics(lda, tf_feature_names, no_top_words)
```

topicmodelling_scikitlearn.py hosted with ❤ by GitHub view raw

Machine Learning | Data Science | Scikit Learn | Topic Modeling | Lda

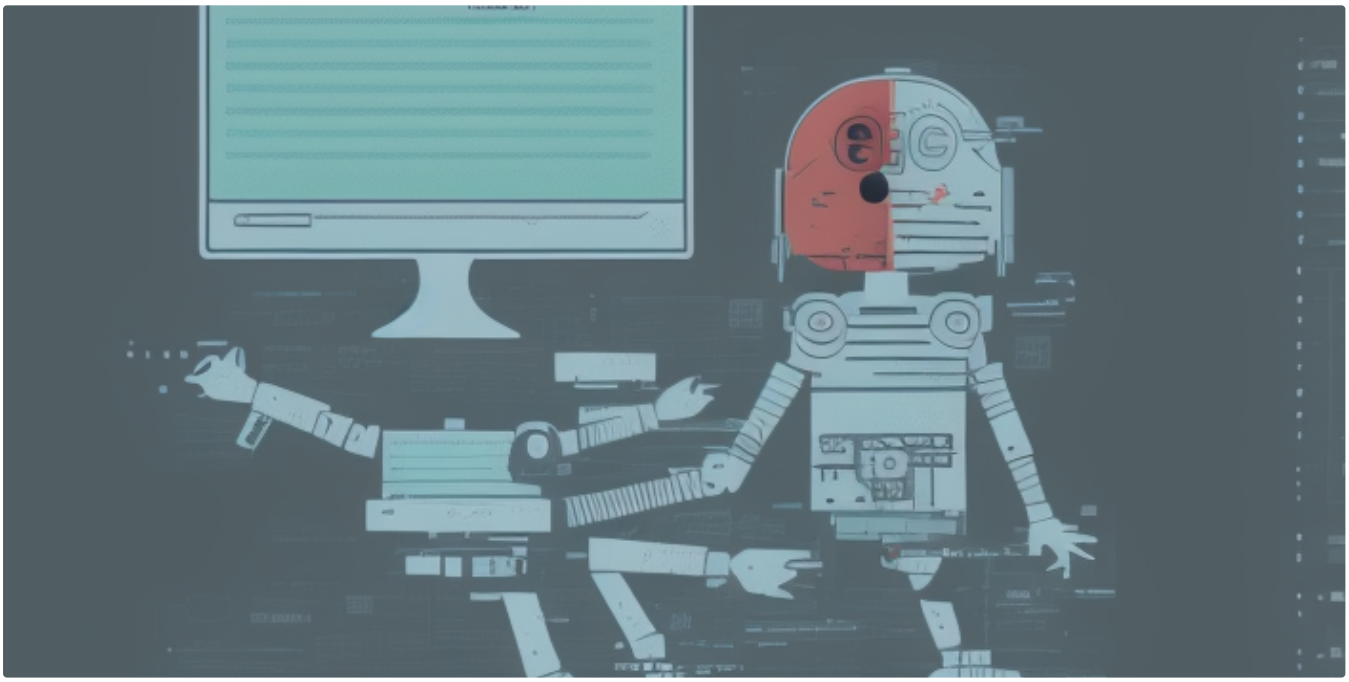
Follow

Written by Aneesha Bakharia

2.4K Followers · Writer for ML Review

Data Science, Topic Modelling, Deep Learning, Algorithm Usability and Interpretation, Learning Analytics, Electronics — Brisbane, Australia

More from Aneesha Bakharia and ML Review



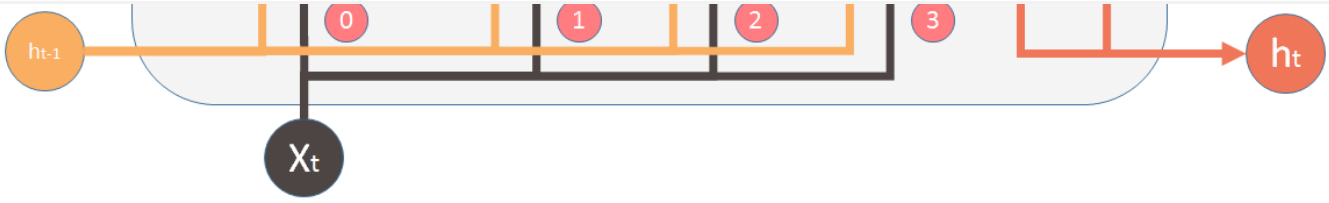
Aneesha Bakharia

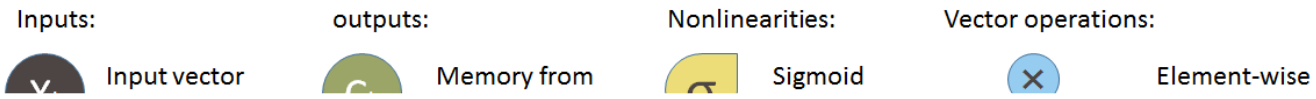
ChatGPT and Markdown formats—Generating all Sorts of Editable Diagrams and Formats

This is a rapidly pulled-together blog post, mainly to share the results of getting ChatGPT to output Markdown. Markdown can be edited and...

4 min read · Jan 11

114 3





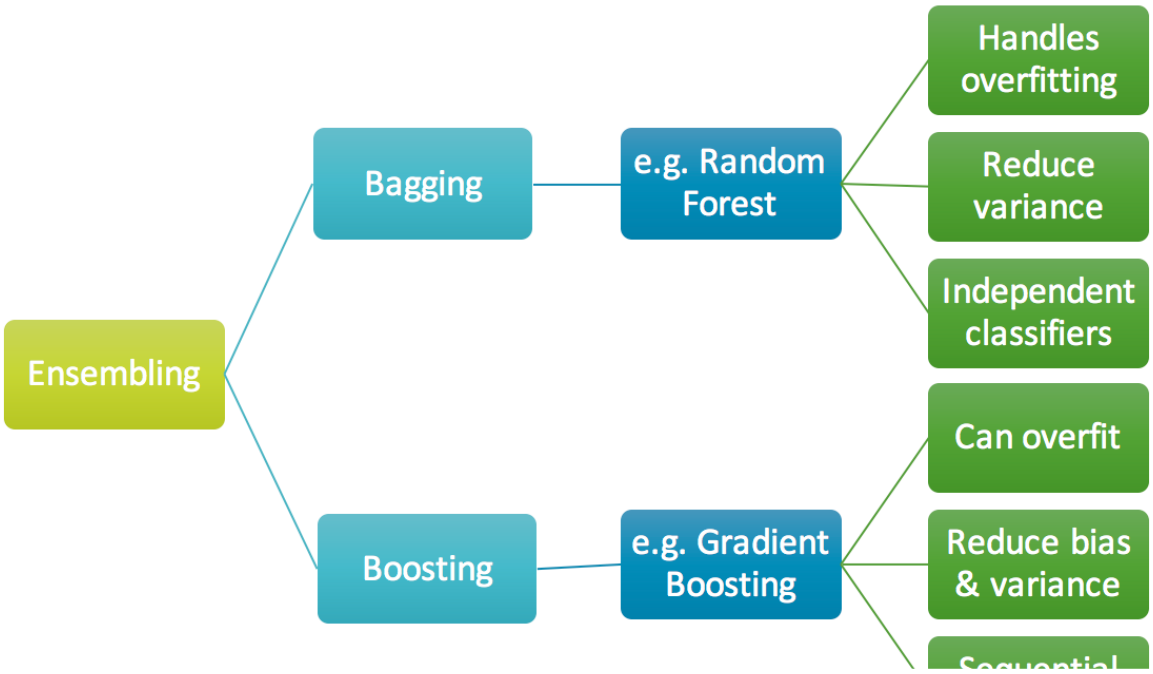
Shi Yan in ML Review

Understanding LSTM and its diagrams

I just want to reiterate what's said here:

7 min read · Mar 14, 2016

9.7K 62



Prince Grover in ML Review

Gradient Boosting from scratch

Simplifying a complex algorithm

8 min read · Dec 9, 2017

9.9K 30

Aneesha Bakharia in Towards Data Science

Quick Semantic Search using Siamese-BERT Networks

Image by PDPhotos and downloaded from Pixabay.

2 min read · Jan 19, 2020

292 2

See all from Aneesha Bakharia

See all from ML Review

Recommended from Medium

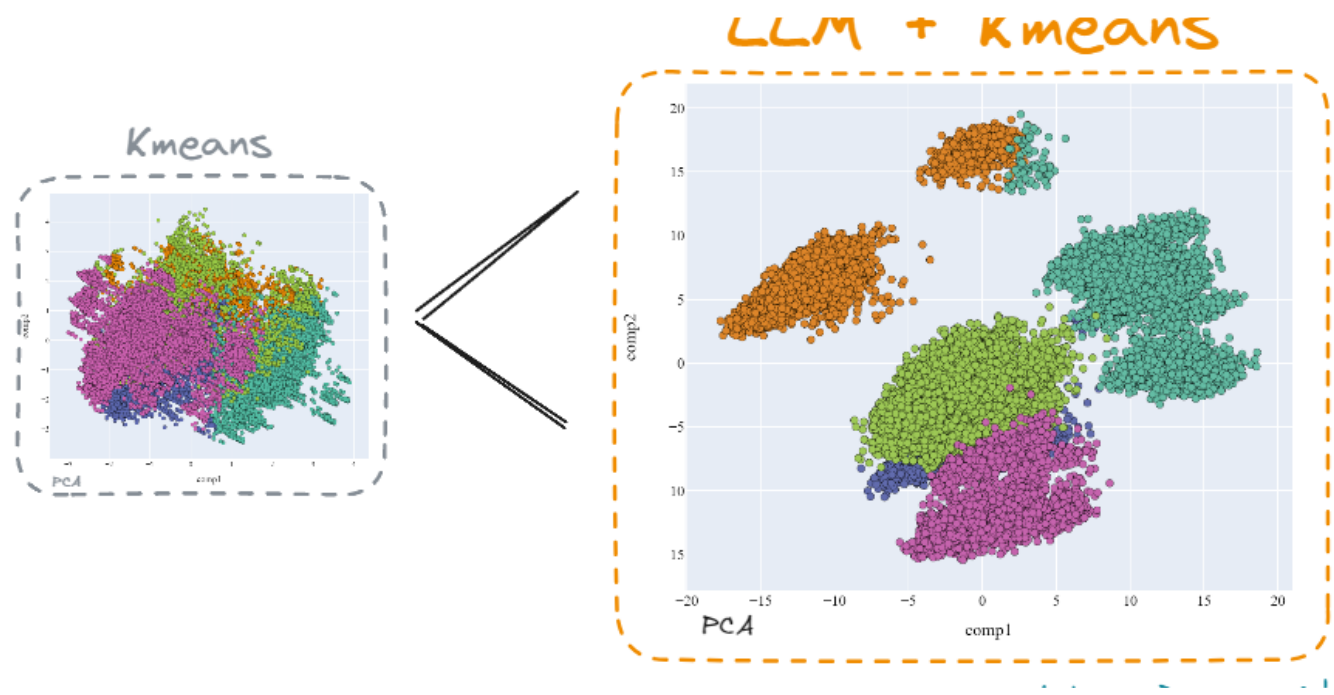
Jawwad Shadman Siddique

Topic Modeling Using BERTopic on Newsgroup Dataset: Python Implementation

We go step by step from creating a google collab workspace to visualizing the cluster of topics in a group of text documents. The...

7 min read · Jul 5

8



Damian Gil in Towards Data Science

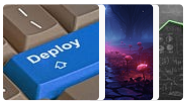
Mastering Customer Segmentation with LLM

Unlock advanced customer segmentation techniques using LLMs, and improve your clustering models with advanced techniques

23 min read · Sep 27

3.1K 25

Lists



Predictive Modeling w/ Python
20 stories · 488 saves

Practical Guides to Machine Learning
10 stories · 557 saves

Natural Language Processing
702 stories · 311 saves

New_Reading_List
174 stories · 149 saves

Stefan Neefischer

Python script: Cluster keywords into topics using SERP results

We've published a Python Script that uses the clustering method to group keywords together using Google's search results. The new version...

3 min read · May 15

2

Daria Sukhareva in GoPenAI

Sentiment analysis per topic modelled in product reviews with ChatGPT [P2]

This article is Part Two of sentiment analysis per topic modelled in product reviews with ChatGPT. In Part 1 we prompted ChatGPT to...

4 min read · May 5

17

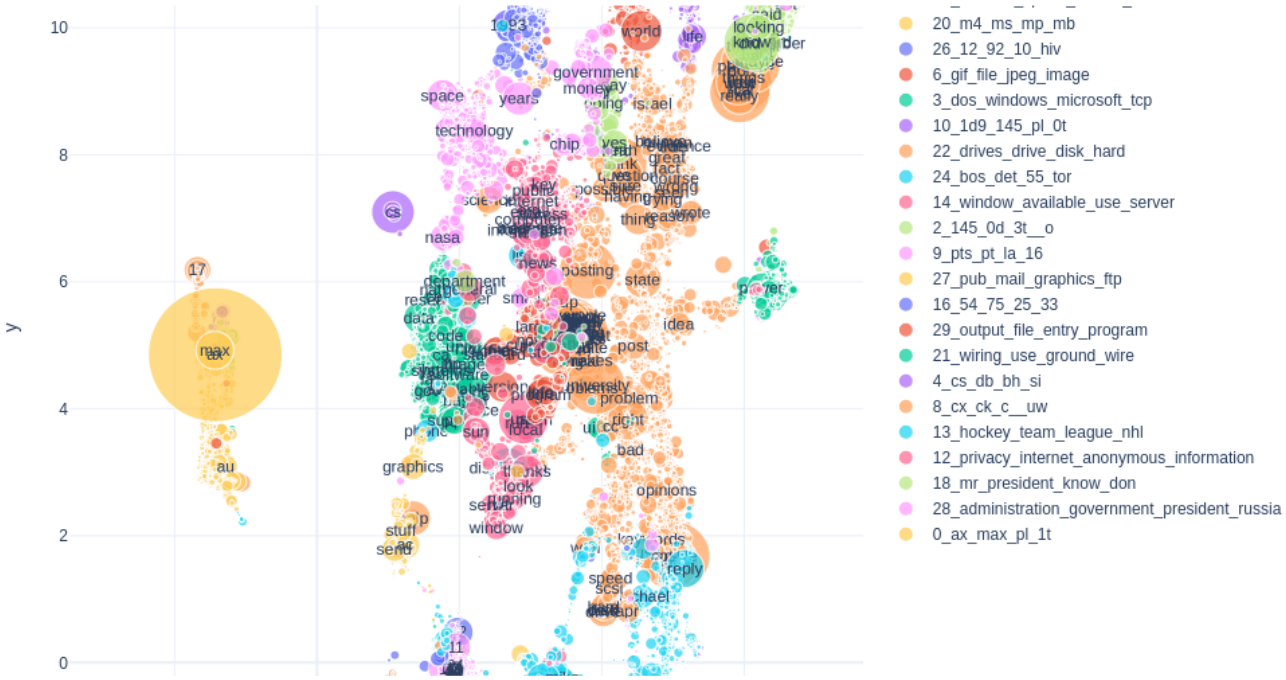
Fruit	Apple	Banana	Orange	Mango	Vector
Apple	1	0	0	0	[1, 0, 0, 0]
Banana	0	1	0	0	[0, 1, 0, 0]
Orange	0	0	1	0	[0, 0, 1, 0]
Mango	0	0	0	1	[0, 0, 0, 1]

Nimrita Koul

Natural Langauge Processing with Python Part 4: Text Representation

This article is the fourth in the series of my articles covering the sessions I delivered for WomenWhoCode datascience track event in March...

9 min read · Apr 28



Márton Kardos

Unsupervised Text Classification with Topic Models and Good Old Human Reasoning

Use your brain and your data interpretation skills, and create production-ready pipelines without labeled data

9 min read · Aug 3

 21  1